La App que Nunca se Traba

Pedro Luis Alvarez Gonazalez
7690-22-5839
Universidad Mariano Gálvez
Seminario de Tecnologías de Información
palvarezg1@miumg.edu.gt

Resumen

El objetivo de este trabajo es explicar, de forma clara, cuáles son las buenas prácticas que debe seguir un desarrollador al hacer aplicaciones reactivas, es decir, aplicaciones que responden rápido, aguantan carga y no se caen fácil cuando algo falla. Para lograrlo se investigó sobre que es Reactivo, sobre el manifiesta Reactivo y sobre patrones de diseño reactivo usados en nuestro ambito de desarollo. Se ordenó la información en principios prácticos que cualquier programador puede aplicar: programación asíncrona y no bloqueante, manejo correcto del backpressure, aislamiento de fallos, inmutabilidad de datos y monitoreo constante del sistema. Como resultado se armó una guía que conecta la teoría del Manifiesto Reactivo con decisiones concretas de código y arquitectura. Se concluye que una aplicación reactiva no se logra solo usando una librería de moda, sino aplicando una forma distinta de pensar el flujo de datos y los errores dentro del sistema desde el inicio del desarrollo.

Palabras clave: programación reactiva, sistemas asíncronos, resiliencia, backpressure, arquitectura de software.

Desarrollo del tema

Una aplicación reactiva es aquella que está diseñada para responder rápido, mantenerse disponible aunque algo falle y adaptarse a la carga de trabajo sin caerse. Esta forma de construir software se basa en cuatro principios que plantea el Manifiesto Reactivo: que el sistema sea responsivo es decir responde rápido siempre, resiliente es que sigue funcionando aunque una parte falle, elástico que aguanta más o menos carga según se necesite y orientado a mensajes es decir los componentes se comunican de forma asíncrona en vez de estar esperándose unos a otros.

La primera buena práctica y probablemente la base de todo, es programar de forma asíncrona y no bloqueante. Esto quiere decir que el código no debe quedarse esperando a que una operación termine, como una consulta a base de datos o una llamada a otro servicio, sin hacer nada mientras tanto. En vez de eso, se usan callbacks, promesas o streams reactivos para que la cola de ejecución quede libre para hacer otras tareas mientras la operación lenta se resuelve en segundo plano. Bloquear una cola esperando una respuesta es de los errores más comunes que hacen que una aplicación reactiva en realidad no lo sea.

La segunda práctica importante es el manejo correcto del backpressure. Cuando un componente produce datos más rápido de lo que otro los puede consumir, si no se controla esto, el sistema se satura y puede colapsar. Las especificaciones sirven justamente para que el consumidor le pueda avisar al productor cuánta información puede recibir en un momento dado, evitando que se sature la memoria o se caiga el sistema completo.

La tercera práctica es diseñar pensando en el aislamiento de fallos. En un sistema reactivo, un componente no debe caer todo el sistema si él falla. Esto se logra separando responsabilidades en componentes pequeños e independientes y aplicando patrones que corten la comunicación, con un servicio que está fallando en vez de seguir insistiendo, empeorando el problema y los tiempos de espera crecientes, que reintenta una operación fallida sin saturar el sistema.

La cuarta práctica tiene que ver con la inmutabilidad de los datos. En sistemas donde varias partes del código se ejecutan al mismo tiempo, conocida como concurrencia, si dos procesos modifican el mismo dato al mismo tiempo se pueden generar errores muy difíciles de detectar. Por eso se recomienda trabajar con datos inmutables, es decir, que una vez creados no cambian. Si se necesita una versión distinta, se crea un nuevo dato en vez de modificar el original. Esto reduce muchísimo los bugs relacionados con concurrencia.

La quinta práctica es el monitoreo constante del sistema. Como las aplicaciones reactivas suelen estar compuestas por varios componentes que se comunican de forma asíncrona, es más difícil darse cuenta cuándo algo está fallando si no se tiene visibilidad clara de lo que pasa internamente. Por eso se recomienda implementar logs estructurados y trazabilidad de las peticiones desde el inicio del proyecto, y no dejarlo para el final.

Por último, es importante escoger la herramienta correcta según el lenguaje y el tipo de proyecto, en vez de usar una librería reactiva solo porque está de moda o una inteligencia artificial te lo diga. Estos frameworks existen para resolver problemas distintos y usar el equivocado puede complicar el código en vez de simplificarlo.

Observaciones y comentarios

Durante esta investigación se pudo notar que muchos desarrolladores confunden programación reactiva con simplemente usar una librería asíncrona, sin realmente aplicar los principios de resiliencia y elasticidad que exige el Manifiesto Reactivo. Esto genera aplicaciones que técnicamente usan streams o callbacks, pero que se siguen cayendo igual cuando hay mucha carga o cuando un servicio externo falla. También se observó que el mayor reto no es tanto el código, sino el cambio de mentalidad. Pensar en flujos de datos y eventos en vez de pensar en instrucciones que se ejecutan una tras otra.

Conclusiones

1. Una aplicación reactiva se construye aplicando los cuatro principios del Manifiesto Reactivo que son: responsiva, resiliente, elástica y orientada a mensajes, no solo usando una librería específica.

2. Programar de forma asíncrona y no bloqueante es la base de cualquier sistema reactivo bien hecho.

3. Manejar el backpressure y separar los fallos evita que un problema pequeño colapse todo el sistema.

4. La inmutabilidad de datos y el monitoreo constante son prácticas que reducen errores difíciles de detectar en sistemas.

Referencias

Kuhn, R., Hanafee, D., & Allen, J. (2017). *Reactive design patterns*. Manning Publications.

Reactive Manifesto. (2014). *The reactive manifesto*. https://www.reactivemanifesto.org

Roestenburg, R., Bakker, R., & Williams, R. (2016). *Akka in action*. Manning Publications.

Repositorio: https://github.com/PLAlvarez11/seminario-encuesta.git